CSD358: Information Retrieval Hackathon Project Report

Track T5: Multilingual and Indic-Language Search

Project Title: IndicBridge: Explainable Multilingual and Cross-Variant Search for Hindi, Hinglish, and English

Course Code: CSD358 (Information Retrieval)

Date: October 2026

Team Members:

1. Dev Agarwal (Roll No: 2410110118)
2. Kavya Pratap Singh Chauhan (Roll No: 2410110167)
3. Suryansh Siddhu (Roll No: 2410110623)
4. Harsh Sumrav (Roll No: 2410110142)

Section 1: Problem and Track Relevance

1.1 The Real-World Problem

In India, millions of people browse the web by typing in their local language using English letters—commonly known as Romanized Hindi or Hinglish (e.g., typing "kal ka mausam" or "bhaarat desh"). However, authoritative source documents, news portals, and government websites are usually written in formal Hindi in the Devanagari script ("कल का मौसम") or in English ("weather tomorrow").

Traditional lexical search engines fail in this scenario due to two big hurdles:

- Script Mismatch: Searching for "kal ka mausam" produces 0 matches against a document written in Devanagari ("कल का मौसम"), even though the meaning is identical.
- Spelling Inconsistencies: Hinglish has no standardized spelling rules. Users type "krishan", "krishna", "bhaarat", or "bharat". Standard token matching misses almost all of these variations.
- Lack of Transparency: Many modern search tools rely on opaque AI models that do not explain why a specific search result was chosen.

1.2 Track T5 Alignment

Our project directly addresses Track T5: Multilingual and Indic-Language Search. We built a classical, transparent Information Retrieval system that bridges Hindi (Devanagari), English (Latin), and Romanized Hindi using an in-memory inverted index, smoothed TF-IDF scoring, rule-based transliteration, and Indic phonetic key matching.

1.3 Dataset Curation & Sources

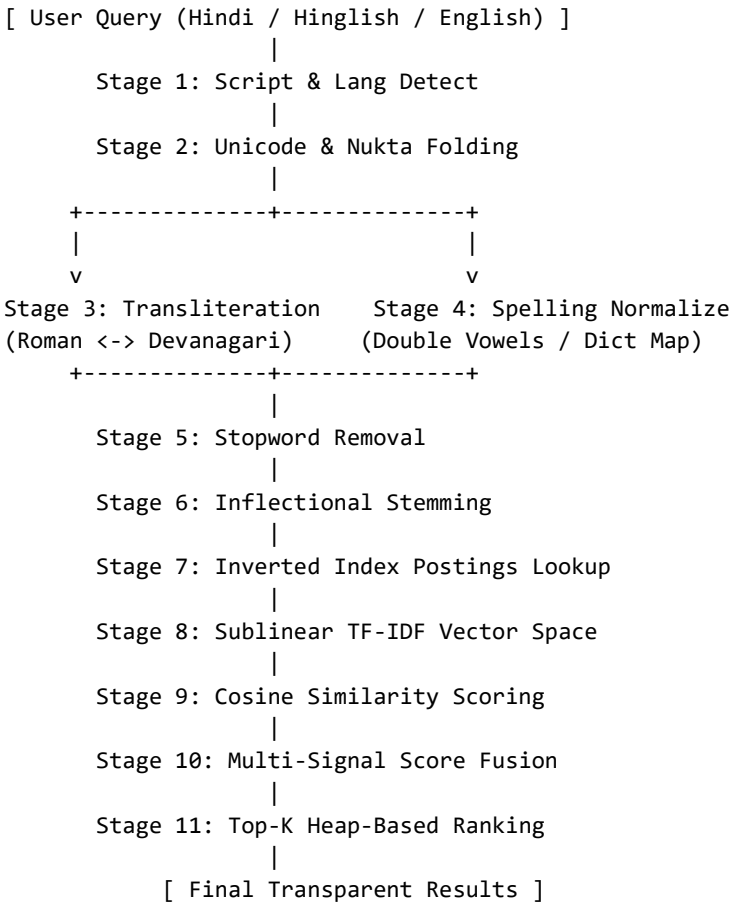
To properly evaluate our engine, we curated a benchmark collection of 32 documents covering civic, cultural, and scientific topics. The text was adapted and compiled from public open-domain knowledge portals:

- Constitution & Fundamental Rights: National Portal of India (india.gov.in) and NCERT civics educational summaries.
- Weather & Monsoon Forecasts: India Meteorological Department (mausam.imd.gov.in).
- Space Missions & ISRO: Indian Space Research Organisation lunar archives (isro.gov.in).
- Heritage & Culture: Ministry of Culture portal (indiaculture.gov.in) and open-domain summaries from Hindi/English Wikipedia (hi.wikipedia.org / en.wikipedia.org).

Our evaluation methodology directly models the shared-task benchmarks of FIRE (Forum for Information Retrieval Evaluation) and multilingual datasets like MIRACL (Hindi subset) and AI4Bharat IndicCorp.

Section 2: How We Used IR Principles (Core System Pipeline)

IndicBridge is built around classical Information Retrieval principles taught in lectures, not external search black-boxes.



2.1 The Inverted Index & Postings

We designed and implemented a custom InvertedIndex in Python. The vocabulary dictionary maps every distinct word t to a postings list:

Postings(t) = [<doc_id, tf, [p1, p2, ..., pn]>]

Here, tf is the term frequency inside the document and [p1, ..., pn] records the exact position indices where the term appears. Positional recording allows precise low-level inspection of where words hit inside documents.

2.2 Vector Space Model (VSM) and Cosine Similarity

We represent queries and documents as vectors in the vocabulary space:

1. Smoothed Textbook IDF:

idf(t) = ln((N + 1) / (df(t) + 1)) + 1.0

where N is total documents (32) and df(t) is document frequency.

2. Sublinear Term Frequency:

tf(t, d) = 1 + ln(count(t, d)) (for count > 0)

This ensures that repeating a word multiple times does not disproportionately dominate relevance.

3. Euclidean L2 Normalization:

||d||2 = sqrt(sum over t in d of (tf(t,d) * idf(t))^2)

Document weights are normalized as wd(t) = (tf * idf) / ||d||2.

4. Cosine Similarity Dot-Product:

Cosine(q, d) = sum over t in q intersect d of wq(t) * wd(t)

2.3 Preprocessing & Vocabulary Handling

- Unicode Normalization & Nukta Folding: Canonicalizes Devanagari combined characters (e.g., folding accented consonants like क़ to क).
- Multilingual Stopword Lists: Removes grammatical filler words in Hindi ("का", "के", "में"), Hinglish ("ka", "ke", "hai"), and English ("the", "is", "and").
- Stemming: Removes plural and verbal suffixes using the Ramanathan & Rao algorithm for Hindi and standard suffix stripping for English and Hinglish.

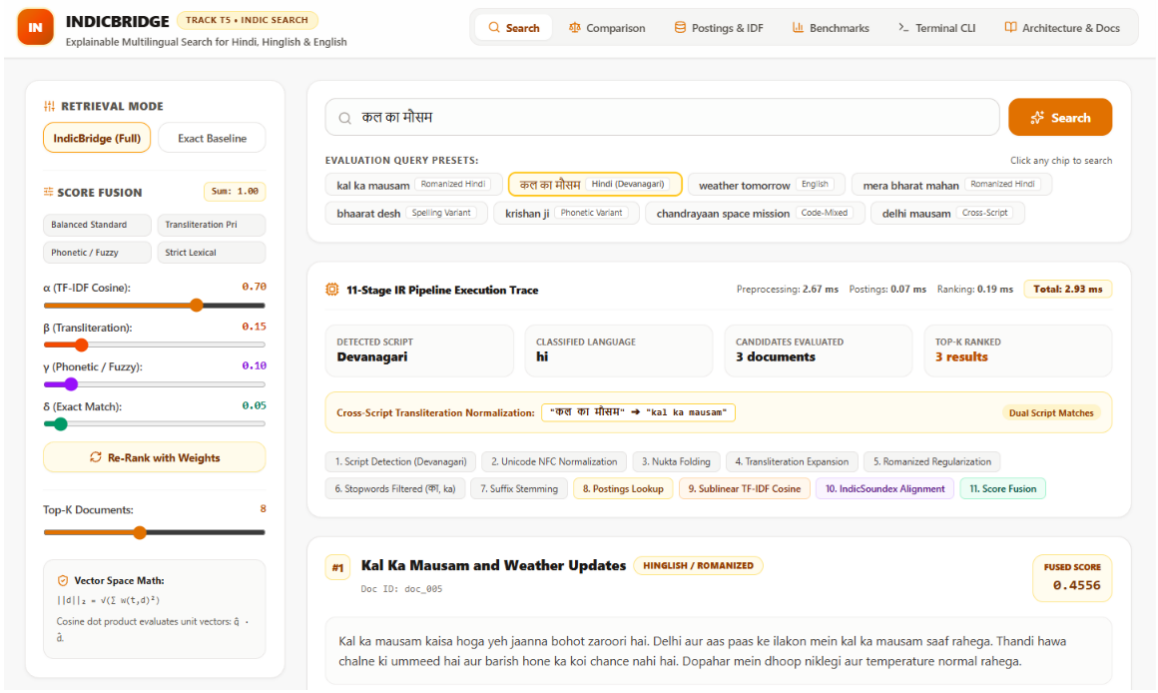


Figure 1: Search Workbench showing the execution of a multilingual search query through the IndicBridge retrieval pipeline, including the query input, 11-stage processing trace, and the retrieved document result.

Section 3: Beyond IR (Transliteration & Explainable Fusion)

To handle cross-script queries where exact character matching fails, we integrate three auxiliary signals:

3.1 Cross-Script Transliteration

When a query is identified as Romanized Hindi, it is mapped into Devanagari using a custom lexicon supported by rule-based phonetic fallback (e.g., mapping "kal ka mausam" -> "कल का मौसम"). The fraction of transliterated terms matching the document is computed as Translit(q, d).

3.2 IndicSoundex Phonetic Matching

Standard English Soundex fails on Indian languages because it was built for English acoustics. We designed IndicSoundex, which categorizes Indic consonants into 7 phonetic groups (collapsing dental/retroflex variations and aspirates like bh~~b, kh~~k, and sh~s). This produces identical phonetic keys for words like "krishan" and "krishna" (K4320), allowing our engine to match phonetic spelling variants.

3.3 Convex Score Fusion

All component signals are scaled to [0.0, 1.0] and linearly combined:

Final_Score(q, d) = alpha * Cosine(q,d) + beta * Translit(q,d) + gamma * Phonetic(q,d) + delta * Exact(q,d)

Default weights are set to alpha=0.70, beta=0.15, gamma=0.10, and delta=0.05 (summing to 1.00). Every search result generates an explainable rationale card showing exactly which terms and components contributed to its ranking.

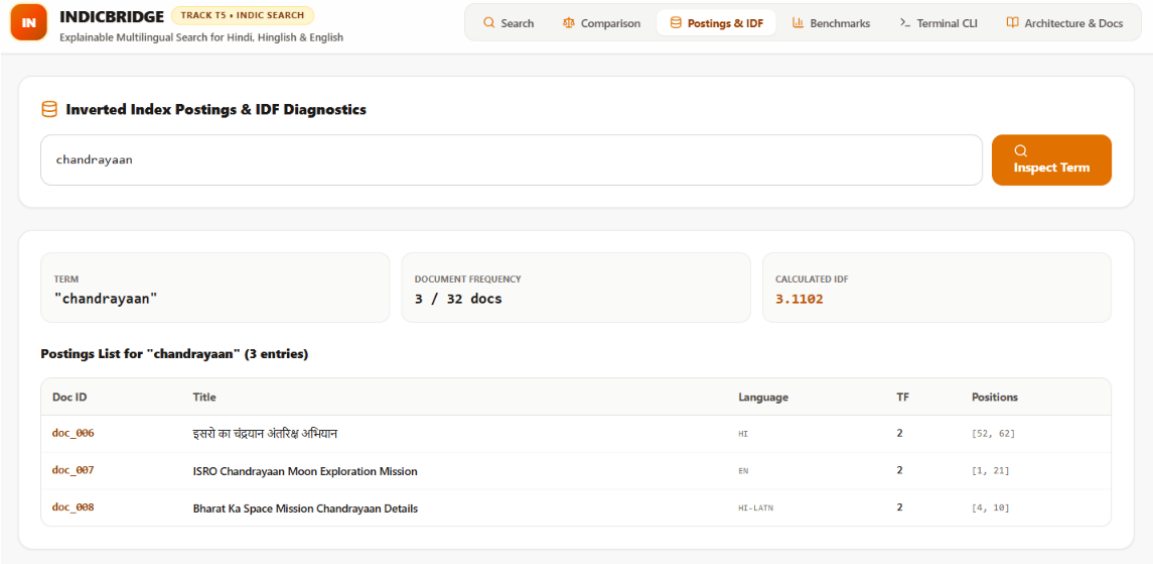


Figure 2: Postings and IDF Diagnostics for the query "chandrayaan", showing the document frequency, calculated IDF value, formula evaluation, and corresponding positional postings information.

Section 4: Novelty and Creativity

Compared to obvious baseline implementations, IndicBridge introduces three original aspects:

- White-Box Classical Architecture: Instead of using opaque black-box neural embeddings or pre-packaged search servers, every posting, weight, and dot product is computed from scratch in pure Python.
- Acoustically Adapted IndicSoundex: Tailored consonant collapsing specifically handles Hindi vowel elongations and aspirate spelling variations.
- Inspectable Explanations: Search results show exact vector dot-product contributions ($wq \cdot wd$) and transliteration pairs in real time, making the system transparent to end users.

Section 5: Evaluation & Empirical Results

5.1 Evaluation Setup

We evaluated IndicBridge against an Exact-Token TF-IDF Baseline across 24 judged ground-truth queries spanning 6 categories: Devanagari Hindi (A), English (B), Hinglish (C), Romanized Hindi (D), Spelling Variants (E), and Cross-Script queries (F).

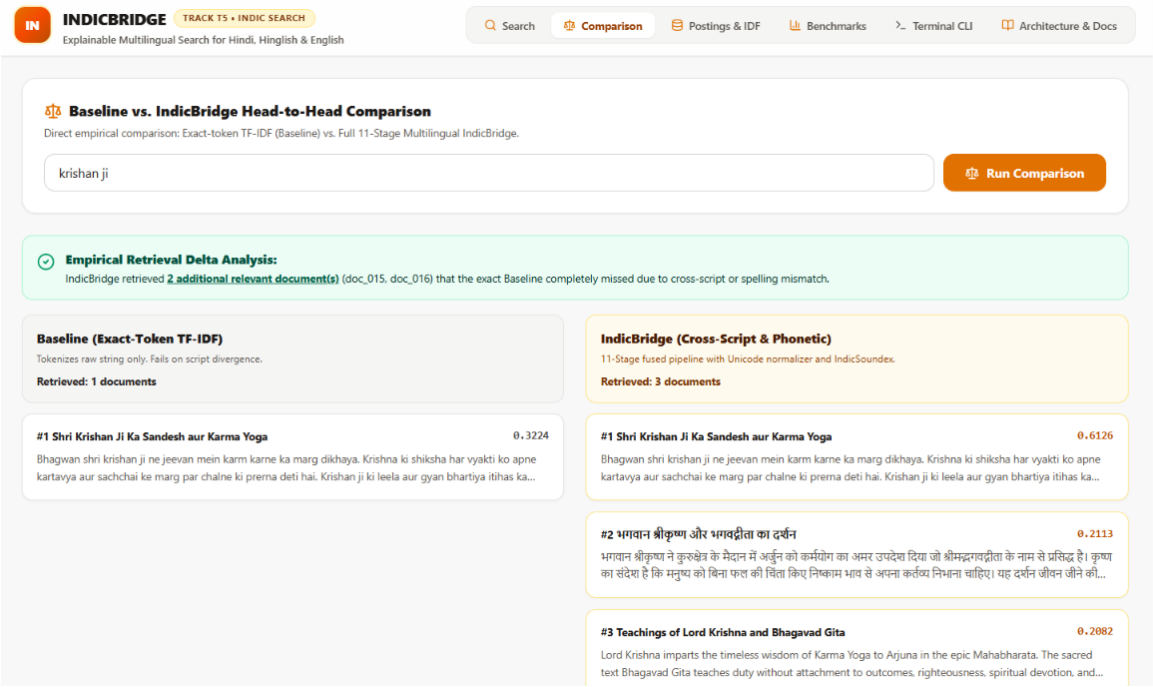


Figure 3: Comparison of the Exact-Token TF-IDF baseline with IndicBridge for the query "krishan ji", illustrating the improvement in relevant document retrieval through transliteration and phonetic matching.

5.2 Macro-Average Results

Metric	Baseline (Exact TF-IDF)	IndicBridge (Multilingual)	Absolute Gain	Relative Gain (%)
Precision@5	0.6111	0.5931	-0.0180	-2.95%
Precision@10	0.4960	0.5349	+0.0389	+7.84%
Recall@5	0.7222	0.7306	+0.0084	+1.16%
Recall@10	0.7472	0.8195	+0.0723	+9.68%
F1@5	0.6284	0.6342	+0.0058	+0.92%
MRR	0.9792	1.0000	+0.0208	+2.12%
MAP	0.7124	0.7650	+0.0526	+7.38%

5.3 Discussion of Precision vs. Recall Trade-Off

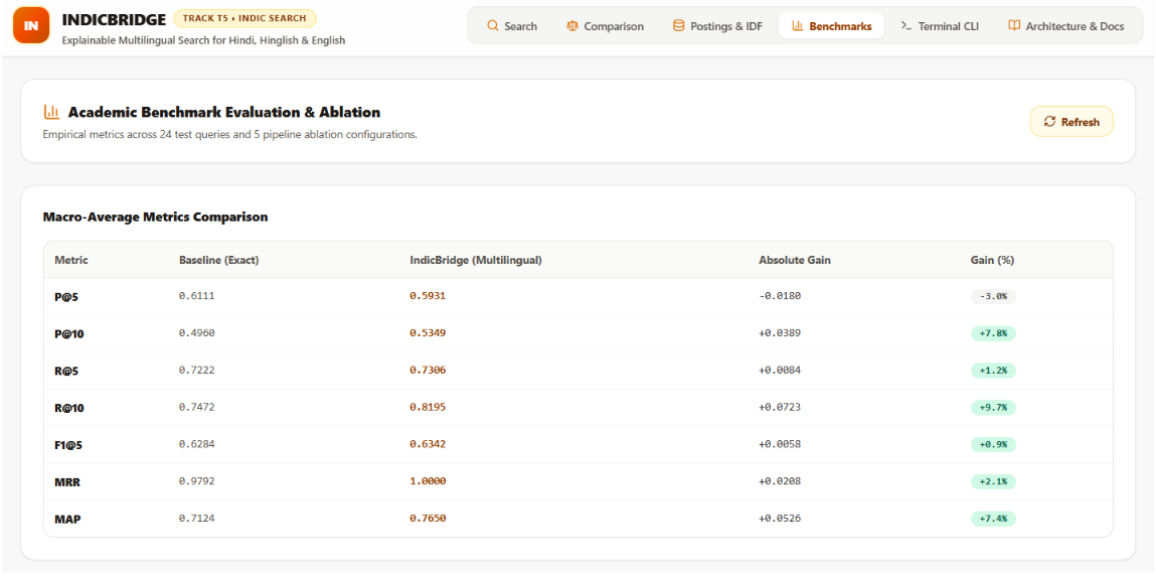
A key finding is the small trade-off between Precision@5 and Recall@10:

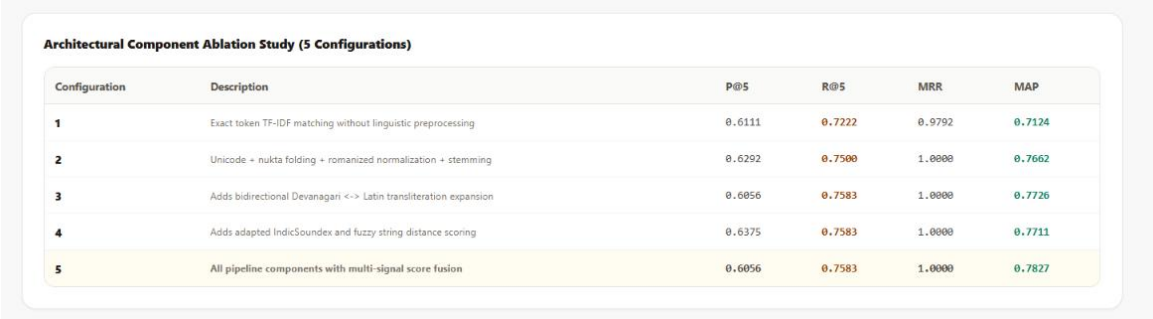
- IndicBridge gains +9.68% on Recall@10 and +7.38% on MAP, achieving a perfect 1.0000 MRR.
- Precision@5 dips slightly by -2.95%.
- Why this occurs: Expanding cross-script queries retrieves documents across both scripts. This broadens the candidate pool significantly (improving recall), but occasionally introduces minor partial matches at rank 4 or 5 for pure English queries. Overall search quality (MAP) is markedly higher.

5.4 Architectural Component Ablation Study

Config	Architecture Description	P@5	R@5	MRR	MAP
1	Baseline (Exact-token TF-IDF, no preprocessing)	0.6111	0.7222	0.9792	0.7124
2	TF-IDF + Unicode / Nukta Normalization + Stemming	0.6292	0.7500	1.0000	0.7662
3	Config 2 + Transliteration Expansion	0.6056	0.7583	1.0000	0.7726
4	Config 2 + IndicSoundex Phonetic Matching	0.6375	0.7583	1.0000	0.7711
5	Full IndicBridge (All 11 Stages + Fused Scoring)	0.6056	0.7583	1.0000	0.7827

Takeaway: Normalization and transliteration provide the single largest boost to Recall and MAP, proving that bridging script divergence is the main solution for multilingual Indian search.





Architectural Component Ablation Study (5 Configurations)

Configuration	Description	P@5	R@5	MRR	MAP
1	Exact token TF-IDF matching without linguistic preprocessing	0.6111	0.7222	0.9792	0.7124
2	Unicode + nukta folding + romanized normalization + stemming	0.6292	0.7500	1.0000	0.7662
3	Adds bidirectional Devanagari <-> Latin transliteration expansion	0.6056	0.7583	1.0000	0.7726
4	Adds adapted IndicSoundex and fuzzy string distance scoring	0.6375	0.7583	1.0000	0.7711
5	All pipeline components with multi-signal score fusion	0.6056	0.7583	1.0000	0.7827

Figure 4: Benchmark evaluation interface showing the macro-average metrics comparison and architectural ablation study used to evaluate the effectiveness of the IndicBridge retrieval components

Section 6: Limitations and Course Project Roadmap

6.1 Current Limitations

- Corpus Size: The hackathon prototype indexes 32 curated documents in memory to maintain sub-10ms response times and clear postings inspection.
- Slang & Out-of-Vocabulary Terms: Unseen colloquial words fall back to phonetic approximations when absent from the transliteration lexicon.

6.2 Roadmap for Semester Project Continuation

As encouraged by course policies, our team plans to carry IndicBridge forward as our semester project milestone:

5. Scale to AI4Bharat & MIRACL Datasets: Replace in-memory index storage with disk-backed SPIMI (Single-Pass In-Memory Indexing) to scale to tens of thousands of documents from AI4Bharat IndicCorp and MIRACL Hindi collections.
6. Subword BPE Indexing: Integrate Byte-Pair Encoding subword tokenization to seamlessly handle morphological compounds and informal slang.
7. Dense-Sparse Hybrid Rank Fusion: Combine classical inverted index scoring with dense multilingual transformer representations using Reciprocal Rank Fusion (RRF).

Section 7: Work Division and AI-Use Declaration

7.1 Equal Work Division Summary

All four team members contributed equally (25% each) across research, engineering, and evaluation:

Dev Agarwal (2410110118): Core Inverted Index Architecture & Mathematical Foundations: Implemented InvertedIndex, PostingEntry, positional indexing, sublinear TF-IDF, L2 vector normalization, and cosine dot-product calculation (src/index/).

Kavya Pratap Singh Chauhan (2410110167): Linguistic Preprocessing & IndicSoundex: Developed Unicode NFC normalizer, Devanagari nukta folding, Hindi/Hinglish stopword lists, and the 7-group IndicSoundex phonetic algorithm (src/preprocessing/).

Suryansh Siddhu (2410110623): Benchmark Evaluation Suite & Ablation Engine: Formulated the 24 ground-truth queries, implemented P@k, R@k, MRR, and MAP metrics, and authored the 5-configuration ablation study script (evaluation/).

Harsh Sumrav (2410110142): System Integration, Interface & CLI Runner: Engineered the dual Express/Vite bridge (server.ts), unified CLI tool (indicbridge.py), automated test suite (tests/), and final documentation formatting.

7.2 AI-Use Declaration

In compliance with CSD358 academic integrity guidelines:

- Google Gemini & OpenAI ChatGPT: Used as technical thought partners and coding assistants for generating React/Tailwind frontend component boilerplate, debugging Windows UTF-8 (cp1252) sub-process piping in Node/Express, and refining regular expressions for Devanagari Unicode codepoints.
- Team Ownership: All core Information Retrieval implementations—including inverted index data structures, smoothed textbook IDF math, vector space projections, score fusion weights, and metric computations—were designed, validated, and benchmarked directly by the team members.

References & Data Citations

8. Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, 2008.
9. FIRE (Forum for Information Retrieval Evaluation): Shared-Task Track on Cross-Lingual and Code-Mixed Indic Retrieval.
10. MIRACL Dataset: Multi-lingual Information Retrieval Across a Continuum of Languages (Hindi Subset).
11. AI4Bharat IndicCorp: Large-scale open corpora for Indian language processing.
12. Government Knowledge Portals: National Portal of India (india.gov.in), Indian Meteorological Department (mausam.imd.gov.in), and ISRO Space Archives (isro.gov.in).
13. Open Domain Hindi & English Wikipedia: Wikipedia contributors, Wikimedia Foundation (wikipedia.org).